



AULA 3

**Arquitetura, projeto e controle da
qualidade de sistemas de *software*
inteligentes**



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Compreender arquiteturas de sistemas inteligentes com base nas formas de implantação de modelos de *machine learning* mais utilizados no mercado.
- Aprender como princípios de projeto (como SOLID) podem ser aplicados no contexto de *machine learning*.
- Conhecer os conceitos básicos e práticas de controle da qualidade (análise estática, revisão de código e teste).
- Entender os conceitos de gerência de configuração e DevOps (controle de versões, integração contínua e implantação contínua).

Que história é essa?

Nesta terceira aula, iremos compreender arquiteturas de sistemas inteligentes com base nas formas de implantação de modelos de *machine learning* mais utilizados no mercado. Iremos ainda abordar a importância da aplicação de princípios de projeto de *software* no desenvolvimento de soluções de *machine learning* e aprender a realizar essa aplicação na prática. Além disso, vamos conhecer conceitos básicos e práticas de controle da qualidade, gerência de configuração e DevOps.

Arquitetura e implantação de modelos de *machine learning*

A implantação de modelos de *machine learning* em produção envolve disponibilizar modelos treinados, avaliados e aprovados. Na área de ciência de dados, é muito comum que atividades de análise exploratória, pré-processamento, treinamento e avaliação do modelo ocorram utilizando-se programação literária interativa, os famosos *notebooks*. A boa notícia é que, após treinar o modelo, é possível salvá-lo usando a biblioteca **pickle** (p. ex., **model.pkl**) ou salvá-lo como um *script* **.py** (p. ex., **model.py**).

O trecho de código a seguir armazena um modelo treinado que se encontra na variável *modelo* em um arquivo chamado *classificador.pkl* utilizando a biblioteca **pickle**. Note que basicamente foi aberto um arquivo para escrita em formato binário ('wb') e que o método *dump* da biblioteca **pickle** se encarrega de passar o conteúdo do modelo de forma serializada para o arquivo que foi aberto. Depois, basta fechar o arquivo.

```
cod.py

# Salvando o modelo treinado em um arquivo
import pickle
pickle_out = open('classificador.pkl', 'wb')
pickle.dump(modelo, pickle_out)
pickle_out.close()
```

Após a execução desse código em uma célula de um *notebook*, o arquivo *classificador.pkl* existirá na sua máquina, contendo o seu modelo. A partir daí, para carregar o modelo em um sistema que precise realizar uma predição utilizando o modelo, basta disponibilizar o arquivo e integrar no código desse sistema as linhas abaixo. Desta vez, o código envolve a abertura de um arquivo para leitura em formato binário (*rb*), e o método *load* da biblioteca ***pickle*** se encarrega de recuperar o conteúdo do arquivo serializado e de passá-lo para a variável modelo. Por fim, a predição pode ser feita acionando o método *predict* do modelo. Neste exemplo, foi carregado um modelo que faz a classificação de um cliente em um bom ou mau pagador, com base na renda mensal, idade, número de dependentes e valor consignado em empréstimos vigentes.

```
cod.py

# Usando a inferência de um modelo a partir de um arquivo
import pickle
# Carregando o modelo treinado
pickle_in = open('classificador.pkl', 'rb')
modelo = pickle.load(pickle_in)
pickle_in.close()
# Utilizando o modelo para fazer uma predição
Predicao = modelo.predict([[rendaMensal, idade, numeroDependentes, valorConsignado]])
```

Aprenda Mais!

No *link* a seguir, você pode conferir um *notebook* que inclui a construção de um modelo bem simples, sua exportação para o formato pickle e posterior importação e uso.

[Clique aqui](#) para acessar o *notebook* interativo.

Agora que sabemos como armazenar um modelo, podemos pensar na arquitetura para a sua disponibilização. A arquitetura depende fortemente dos requisitos de infraestrutura e da forma de implantação pretendida.

Como vimos anteriormente, quando aprendemos a especificar sistemas de *machine learning*, eles frequentemente impõem requisitos de infraestrutura que podem envolver alto poder computacional para treinamento e inferência. No modelo *serverless*, um provedor em nuvem aloca e fornece dinamicamente os recursos necessários.

Há três formas principais de implantação de modelos de *machine learning*. Confira a seguir:

Modelo “embarcado”

Modelo implantado em um serviço separado

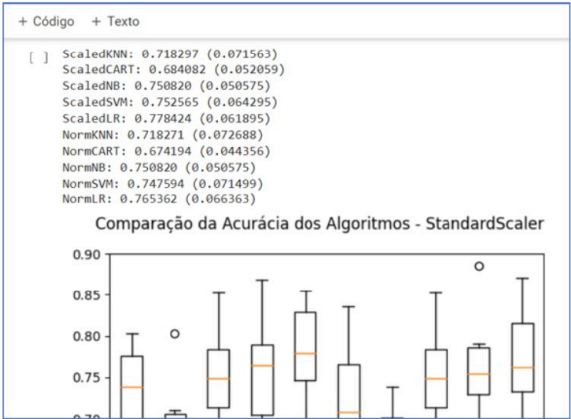
Nesta forma de implantação, o modelo é empacotado e consome. Por exemplo, em aplicações web, o código do *back-end* da aplicação, que também irá quando solicitado pelo usuário (por exemplo, em

Interativo

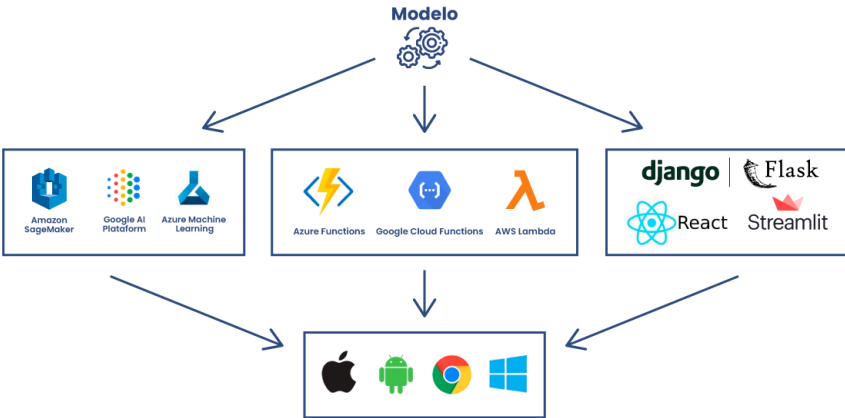
Descrição do interativo

A figura a seguir ilustra as três formas de implantação.

É comum que a análise exploratória, o pré-processamento, o treinamento e a avaliação ocorram em notebooks.



Uma vez que se chega a um modelo adequado, ele é implantado utilizando-se uma das opções de implantação, ficando disponível para ser consumido por aplicações finais que podem ser executadas em diferentes tipos de dispositivos.



Ainda no contexto de arquiteturas de *machine learning*, é comum que ferramentas interativas para visualização de dados sejam empregadas, que permitem gerar *dashboards* (painéis de visualização). Exemplos dessas ferramentas são Power BI, Spotfire e Tableau. Nesse caso, as previsões do modelo podem ser escritas em um banco de dados para permitir gerar visualizações.

Na figura a seguir, temos um exemplo de uma arquitetura que utiliza essa forma de visualização e alguns dos demais conceitos apresentados.

Clique no botão e confira os próximos slides.



Interativo

Descrição do interativo

O retreinamento poderá ocorrer de forma automática caso se tenha investido na automação de tarefas normalmente realizadas pelo cientista de dados, como pré-processamento, treinamento e avaliação do modelo. Esses esforços de automação, incluindo a detecção de flutuações no desempenho, fazem parte do *pipeline* de MLOps, permitindo que o modelo de *machine learning* seja reimplantado em operação quando necessário.

Princípios de projeto aplicados a *machine learning*

Assim como a construção de qualquer tipo de código, o código de *machine learning* pode se beneficiar da aplicação de princípios de projeto para permitir a construção de códigos que sejam mais fáceis de entender e manter, que tenham boa qualidade estrutural. Para exemplificar, falaremos dos princípios SOLID, que são amplamente conhecidos e utilizados no mercado para auxiliar a construção de *software* orientado a objetos.

Os princípios SOLID buscam ajudar a realizar um bom projeto orientado a objetos para que um *software* tenha código flexível, escalável, sustentável e reutilizável. SOLID é um acrônimo para cinco princípios:

- S** *Single-responsibility principle* (SRP) – Princípio de responsabilidade única
- O** *Open/closed principle* (OCP) – Princípio do aberto/fechado
- L** *Liskov substitution principle* (LSP) – Princípio da substituição de Liskov
- I** *Interface segregation principle* (ISP) – Princípio de segregação de interface
- D** *Dependency inversion principle* (DIP) – Princípio da inversão de dependência



A seguir, apresentaremos cada um desses princípios com exemplos de aplicação para *machine learning*. Em alguns exemplos, utilizaremos códigos escritos para a biblioteca scikit-learn, que é uma das bibliotecas mais utilizadas para a construção de sistemas de *machine learning* em Python.

Estudo complementar

Se tiver dúvidas e sentir necessidade de complementar este conteúdo, sugerimos que verifique a Disciplina bonus de Programação Orientada a Objetos. Entretanto, nosso propósito aqui é entender a ideia geral dos princípios e nem tanto focar nos detalhes dos códigos.

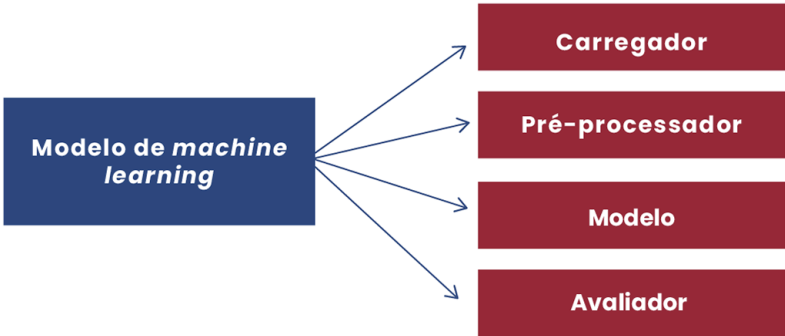
Para acessar a disciplina bonus, vá até a área de entrada do AVA e clique em Programação Orientada a Objetos.

Princípio de responsabilidade única (SRP)

Este princípio diz que uma classe não deve ter mais de um motivo para ser alterada, ou seja, uma classe deve ser especializada em um único assunto e ter apenas uma responsabilidade dentro do *software*. Quando uma classe tem mais de uma responsabilidade e surge uma necessidade de alteração, será difícil modificar uma dessas responsabilidades sem comprometer as outras. Violar esse princípio pode trazer problemas como alto acoplamento, baixa coesão, dificuldades na implementação de testes automatizados e para reaproveitar o código.

Vejamos um exemplo no contexto de *machine learning*. A construção de modelos de *machine learning* normalmente exige um conjunto de atividades a serem realizadas. Entre elas, podemos citar a obtenção dos dados de diferentes fontes, o pré-processamento, a construção do modelo e a avaliação do modelo. Essas atividades podem ser mapeadas em diferentes responsabilidades dentro de um sistema a ser construído.

Poderíamos pensar em ao menos quatro classes com responsabilidades bem definidas e distintas:



A seguir, serão apresentados códigos-exemplo comentados. Note que os códigos foram propositalmente simplificados para que possamos focar em aspectos estruturais na nossa discussão.

O código a seguir apresenta a classe *Carregador*. Ela tem um único método utilitário, chamado *carregar_dados*. Basicamente o que ela faz é internamente chamar o comando *read_csv* da biblioteca

Pandas, retornando um dataframe do Pandas com todos os dados carregados. Claro que essa classe poderia precisar de outros métodos caso a carga de dados fosse proveniente de diferentes fontes, por exemplo. De qualquer forma, essa responsabilidade referente à carga dos dados seria tratada dentro dessa classe.

```
class Carregador:
    def carregar_dados(self, url: str, atributos: list):
        """ Carrega e retorna um DataFrame. Há diversos parâmetros no read_csv que poderiam ser utilizados para dar opções adicionais. """
        return pd.read_csv(url, names=atributos)
```

A classe *PreProcessador*, por sua vez, trata da responsabilidade de pré-processamento. Ela tem um método *pre_processar*, que é responsável pela limpeza dos dados, seleção de atributos (*feature selection*), divisão dos dados em treino e teste e normalização ou padronização dos dados. Seu código se encontra na figura a seguir:

```
class PreProcessador:
    def pre_processar(self, dataset, percentual_teste, seed=7):
        """ Guia de todo o pré-processamento. """
        # limpeza dos dados e eliminação de outliers

        # feature selection

        # divisão em treino e teste
        X_train, X_test, Y_train, Y_test = self.__preparar_holdout(dataset,
                                                                    percentual_teste,
                                                                    seed)

        # normalização/padronização

        return (X_train, X_test, Y_train, Y_test)

    def __preparar_holdout(self, dataset, percentual_teste, seed):
        """ Divide os dados em treino e teste usando o método holdout. Assume que a variável target está na última coluna. O parâmetro test_size é o percentual de dados de teste. """
        dados = dataset.values
        X = dados[:, 0:-1]
        Y = dados[:, -1]
        return train_test_split(X, Y, test_size=percentual_teste, random_state=seed)
```

Para fins de simplificação, a limpeza dos dados e a eliminação de *outliers* não estão implementadas. Normalmente, essa limpeza envolve tratar dados faltantes (por exemplo, substituindo-os pela mediana do restante dos dados, quando pertinente) e eliminar dados que desviam significativamente do comportamento esperado (*outliers*) e que poderiam enviesar o comportamento do modelo a ser construído.

O pré-processamento conceitualmente também envolve a seleção dos atributos (*feature selection*) a serem considerados para a construção do modelo, mantendo somente os atributos que realmente ajudam a construir um modelo que permita boas previsões.

Outro aspecto importante do pré-processamento é a divisão dos dados em treino e teste. Para tratar esse aspecto, no código foi criada uma função não pública chamada *__preparar_holdout* que não é visível externamente (em Python, os métodos não públicos se iniciam com dois underscores, *__*), mas que apoia a separação dos dados de teste. Basicamente o que essa função faz é assumir que a variável alvo está na última coluna, dividir os dados em X (entradas) e Y (saída) e acionar a função *train_test_split* da scikit-learn, que separa os dados X e Y em X de treino e teste e Y de treino e teste, respeitando um percentual fornecido como parâmetro.

Por fim, o pré-processamento poderia ainda aplicar ações de normalização ou padronização dos dados para, então, retornar os conjuntos *X_train*, *X_test*, *Y_train* e *Y_test* já com os devidos tratamentos para que possam ser utilizados para a construção do modelo. É importante ressaltar que este é apenas um exemplo simplificado e que as ações concretas a serem realizadas durante o pré-processamento podem ser definidas com base no conhecimento das técnicas e experimentação para ver quais técnicas efetivamente ajudam a construir modelos melhores. **A aplicação do princípio SRP garante que, caso se precise alterar o pré-processamento, o**

desenvolvedor saberá que essa responsabilidade está sendo tratada dentro dessa classe, tendo um ponto único para modificação dentro do sistema.

O trecho de código a seguir mostra a classe *Modelo*, que tem a responsabilidade da construção do modelo. Nessa implementação, ela tem um único método, chamado *treinar_SVM*, que instancia a classe SVC da biblioteca scikit-learn (que implementa o algoritmo SVM para classificação), treina o modelo com os dados de treino de entrada e de saída (chamando o método *fit* da classe SVC) e retorna o modelo treinado para que ele possa ser utilizado para previsões.

```
cod.py

class Modelo:

    def treinar_SVM(self, X_train, Y_train):
        """ Cria e treina um modelo SVM. Poderia ter um Grid Search
            com cross_validation para escolher os melhores hiperparâmetros, etc.
        """
        modelo = SVC()
        modelo.fit(X_train, Y_train)
        return modelo
```

Novamente fornecemos uma implementação simplificada para que possamos focar didaticamente na estrutura. Existem diversos recursos que podem ser empregados durante o treinamento do modelo, como a validação cruzada e a busca dos melhores hiperparâmetros, por exemplo, utilizando *grid search*. O SVM, por exemplo, tem um parâmetro que poderia ser fornecido na sua instanciação, que é a rigidez da margem (parâmetro C). Quando instanciamos a classe SVC sem fornecer nenhum parâmetro, ele assume o valor *default* C=1. Os códigos referentes à otimização de hiperparâmetros poderiam ficar dentro dessa mesma classe, que tem como responsabilidade realizar o treinamento e a construção do modelo.

Por fim, a classe *Avaliador*, exibida no trecho de código a seguir, cuida da avaliação do modelo utilizando os dados de teste. Basicamente, ela invoca a previsão do modelo para os dados de entrada de teste (*X_test*) e compara os resultados obtidos com os dados de saída de teste (*Y_test*), retornando a acurácia (percentual de acertos). Assim como nos outros exemplos, aqui poderiam ser utilizadas diferentes formas e métricas de avaliação, mas a responsabilidade estaria contida nessa classe.

```
cod.py

class Avaliador:

    def avaliar_acuracia(self, modelo, X_test, Y_test):
        """ faz uma previsão e avalia o modelo. Poderia parametrizar o tipo de
            avaliação, entre outros.
        """
        predicoes = modelo.predict(X_test)
        return accuracy_score(Y_test, predicoes)
```

Uma vez construídas essas classes utilitárias com responsabilidades claramente definidas, elas podem ser instanciadas como no código a seguir:

```
cod.py

# Instanciação das Classes
carregador = Carregador()
pre_processador = PreProcessador()
modelo = Modelo()
avaliador = Avaliador()
```

Utilizando as classes criadas, o código para realizar as responsabilidades de carga de dados, pré-processamento, treinamento e avaliação do modelo se torna extremamente simples, conforme ilustrado a seguir:

26/02/2026, 23:15

Engenharia de software para ciência de dados

```
# Carga
dataset = carregador.carregar_dados(url_dados, atributos)
# Pré-processamento
X_train, X_test, Y_train, Y_test = pre_processador.pre_processar(dataset,
                                                                    percentual_teste)

# Treinamento do modelo
model = modelo.treinar_SVM(X_train, Y_train)
# Impressão do resultado da avaliação
print(avaliador.avaliar_acuracia(model, X_test, Y_test))
```

É possível perceber que, aplicando o princípio SRP, caso adaptações se mostrem necessárias em alguma das responsabilidades, há um ponto de modificação único e de fácil localização (na classe correspondente à responsabilidade). Além disso, melhorias realizadas em uma responsabilidade automaticamente afetarão todas as partes do código que fazem uso da classe correspondente.

Você terá a oportunidade de mergulhar mais nos aspectos de reestruturação para uma boa distribuição de responsabilidades na Técnica Aplicada desta aula.

Princípio do aberto/fechado (OCP)

Este princípio diz que uma classe deve estar aberta para extensão, porém fechada para modificação.



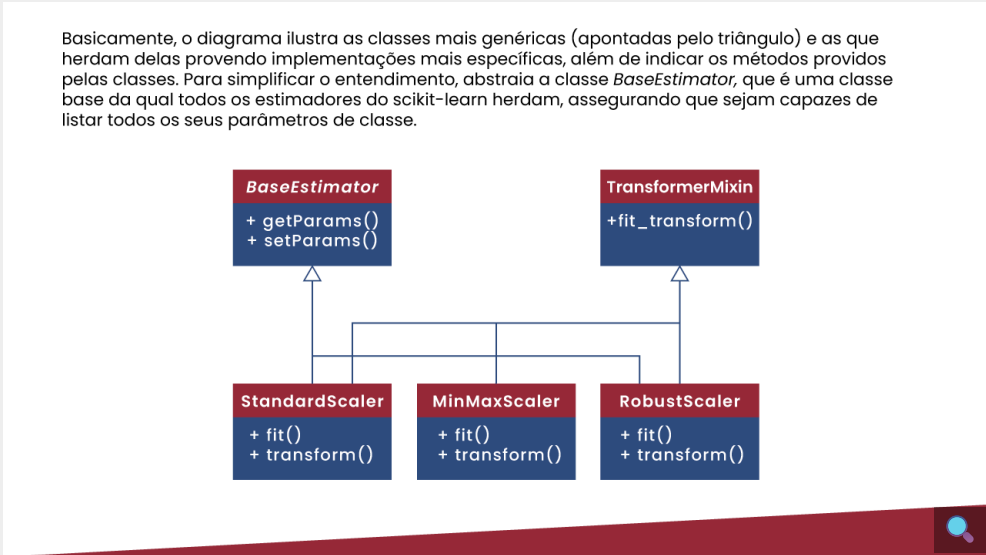
Quando novos comportamentos e recursos precisam ser adicionados no *software*, devemos estender, e não alterar, o código-fonte original. Por exemplo, organizar o seu código com uma superclasse genérica e abstrata o suficiente permitirá que o programa seja estendido adicionando novas subclasses sem precisar alterar as classes existentes.

Diversas aplicações desse princípio podem ser encontradas na própria biblioteca scikit-learn. Por exemplo, temos um conjunto de classes para tratar diferentes transformações de dados durante o pré-processamento, tais como normalização (classe *MinMaxScaler*) e padronização (classe *StandardScaler*). Se todas as diferentes formas de transformação estivessem implementadas em uma única classe, ela teria que ser modificada toda vez que se quisesse acrescentar uma nova forma de transformação de dados à biblioteca.

A solução adotada na scikit-learn foi, de acordo com o princípio OCP, criar uma superclasse mais genérica para permitir que a biblioteca pudesse ser estendida acrescentando novas subclasses, sem precisar alterar as classes existentes. Essa superclasse mais genérica é a classe *TransformerMixin*. Novas classes que buscam realizar transformações sobre dados (assim como a *MinMaxScaler* e a *StandardScaler*) devem herdar dessa classe e implementar os métodos *fit* e *transform*. Isso permitirá a todo o restante da scikit-learn tratar a nova classe de forma genérica como um

TransformerMixin (uma classe que identifica todos os transformadores de dados).

A figura a seguir ilustra um diagrama de classes em *Unified Modeling Language* (UML – a linguagem para modelagem de programas orientados a objetos) ilustrando como três diferentes tipos de transformadores (*StandardScaler*, *MinMaxScaler* e *RobustScaler*) foram criados na scikit-learn herdando das classes mais genéricas *BaseEstimator* e *TransformerMixin*.



O princípio do aberto/fechado (OCP) se manifesta nesse contexto ao assegurar que novos scalers possam ser criados em classes completamente novas, sem afetar o código das demais. Para isso, basta criar uma subclasse que também herde de *TransformerMixin* (e de *BaseEstimator*, como todo estimador da scikit-learn) e que implemente os métodos *fit* e *transform*. Para o restante do sistema, objetos da nova classe serão utilizados de forma transparente como se fossem objetos da classe *TransformerMixin*, e nenhuma outra linha de código terá que ser alterada.

Ou seja, a biblioteca está fechada para modificação (o código anteriormente escrito não precisa ser alterado), mas aberta para extensão. Novas funcionalidades podem ser acrescentadas criando subclasses que terão integração automática no restante do código. Note que, para isso, o restante do código deverá ser construído para depender das abstrações (as classes base da herança), e não das implementações específicas (as subclasses). Programar dependendo das abstrações consiste no princípio da inversão de dependência (DIP), que veremos mais adiante.

Princípio da substituição de Liskov (LSP)

Este princípio diz que uma subclasse deve poder ser substituída pela sua superclasse. Parece estar invertido, uma vez que sabemos que, se S é uma subclasse de C, então os objetos do tipo C podem ser substituídos pelos objetos de tipo S em tempo de execução. No entanto, quando o LSP é seguido, os métodos públicos definidos nas subclasses devem estar também presentes na superclasse, mesmo que de forma abstrata (o que também é conhecido como *strong behavioral subtyping*).

No fundo, o LSP encoraja o uso das classes de nível de abstração mais alto, estruturando as heranças do código de forma a ampliar as possibilidades de aplicação do princípio de inversão de dependência (DIP), uma vez que as abstrações (superclasses) terão os métodos necessários para expressar toda a semântica pretendida.

Fique ligado!

É importante que saibamos quando vale a pena investir neste princípio. Repare que, no exemplo da figura anterior, o princípio LSP não foi seguido, uma vez que a classe *TransformerMixin* não tem os métodos *fit* e *transform*, que aparecem somente nas subclasses. Essa decisão provavelmente foi tomada com base em alguma certeza de que o uso pretendido da superclasse *TransformerMixin* de forma abstrata fosse restrito ao método *fit_transform*, que, pela definição de herança, é herdado também pelas subclasses.

Princípio de segregação de interface (ISP)

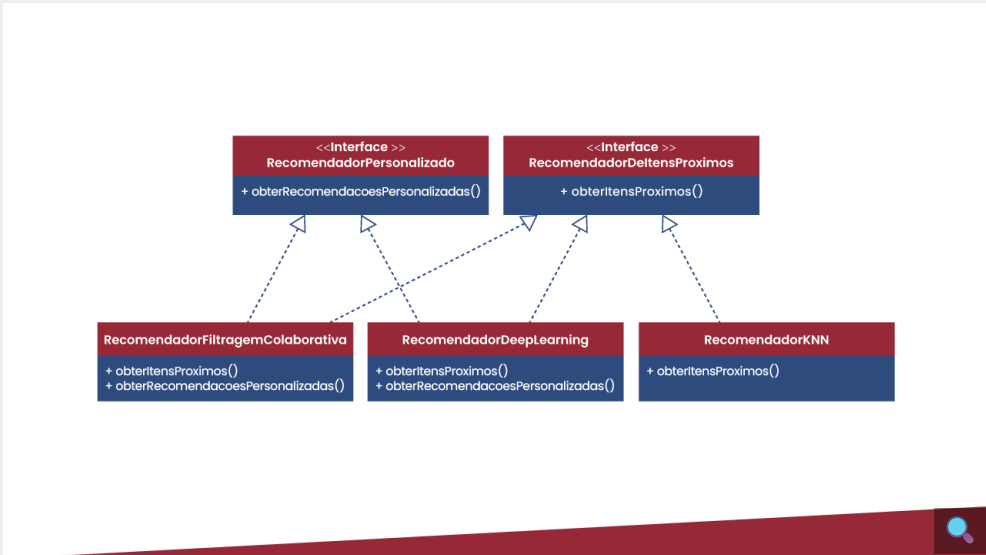
Este princípio define que várias interfaces específicas são melhores do que uma interface genérica, ou seja, que uma classe não deve conhecer nem depender de métodos de que não necessite. Interfaces devem ser segregadas com base nos requisitos em vez de fornecer interfaces grandes de uso genérico.

No exemplo da figura a seguir, representado em UML, a interface *Recomendador* tem dois métodos e três classes que implementam a interface, um recomendador que utiliza filtragem colaborativa, um que utiliza *deep learning* e um que utiliza o algoritmo KNN.

```
classDiagram
    class Recomendador {
        <<Interface>>
        +obterItensProximos()
        +obterRecomendacoesPersonalizadas()
    }
    class RecomendadorFiltragemColaborativa {
        +obterItensProximos()
        +obterRecomendacoesPersonalizadas()
    }
    class RecomendadorDeepLearning {
        +obterItensProximos()
        +obterRecomendacoesPersonalizadas()
    }
    class RecomendadorKNN {
        +obterItensProximos()
        +obterRecomendacoesPersonalizadas()
    }
    RecomendadorFiltragemColaborativa ..|> Recomendador
    RecomendadorDeepLearning ..|> Recomendador
    RecomendadorKNN ..|> Recomendador
```

É possível observar que o recomendador KNN não tem uma implementação para obter recomendações personalizadas, que seriam obtidas com base em padrões comportamentais observados dos usuários. Ele se limita a identificar os itens mais próximos de determinado item de interesse. Entretanto, como ele implementa a interface, ele teria que criar um método para obter recomendações personalizadas, mesmo que sem lógica contida, já que isso está além de sua utilização pretendida.

Nesse caso, seguindo o ISP, seria mais apropriado criar duas interfaces para que cada uma das alternativas de recomendação implemente exatamente as interfaces pretendidas. Isso evitaria que alguém viesse a utilizar o KNN e invocasse o método *obterRecomendacoesPersonalizadas*, comportamento que ele não implementa. A solução está ilustrada a seguir.



O trecho de código complementa a solução e representa a implementação do diagrama UML em Python. Para fins de simplificação, apenas as classes *RecomendadorFiltragemColaborativa* e *RecomendadorKNN* foram implementadas. É possível notar que o *RecomendadorFiltragemColaborativa* implementa as duas interfaces e que, portanto, pode ser utilizado tanto como *RecomendadorPersonalizado* quanto como *RecomendadorDeltensProximos*. É possível observar, ainda, que não

será possível utilizar o *RecomendadorKNN* como um recomendador personalizado, o que, nesse caso, é a solução apropriada.

```
cod.py

class RecomendadorPersonalizado(ABC):
    @abstractmethod
    def obterRecomendacoesPersonalizadas(self, user):
        pass

class RecomendadorDeItensProximos(ABC):
    @abstractmethod
    def obterItensProximos(self, item):
        pass

class RecomendadorFiltragemColaborativa(RecomendadorPersonalizado,
                                         RecomendadorDeItensProximos):

    def obterRecomendacoesPersonalizadas(self, user):
        print("Recomendações Personalizadas.")

    def obterItensProximos(self, item):
        print("Recomendação de itens próximos.")

class RecomendadorKNN(RecomendadorDeItensProximos):

    def obterItensProximos(self, item):
        print("Recomendação de itens próximos.")
```

Princípio da inversão de dependência (DIP)

Este princípio define que devemos depender de abstrações, e não de implementações. Programar para uma interface (a superclasse de maior abstração), e não para uma implementação (subclasse), permitirá desacoplar classes clientes de implementações específicas.

Aprenda Mais!

Um exemplo de aplicação do DIP da biblioteca scikit-learn é a classe *Pipeline*, que permite concatenar um conjunto de transformadores, desde que eles sigam uma interface predefinida. O *pipeline* recebe tuplas de nomes e transformadores (*name, transform*) e, sequencialmente, aplica os transformadores e um estimador final. Os elementos *transform* das tuplas intermediárias do *pipeline* precisam implementar os métodos *fit* e *transform*. O elemento *transform* da tupla final precisa implementar somente o método *fit*.

O código a seguir mostra a construção de um *pipeline* que aplica sequencialmente uma padronização nos dados e depois treina o modelo KNN com os dados já padronizados e imprime os resultados de acurácia obtidos com a predição dos dados de teste. Note que o *pipeline* foi construído com as tuplas (*'StandardScaler', StandardScaler()*) e (*'KNN', KNeighborsClassifier()*), que contêm, respectivamente, instâncias das classes *StandardScaler* e *KNeighborsClassifier*. Ambas as classes têm os métodos *fit* e *transform*. Como a classe *Pipeline* é implementada considerando apenas a abstração, seria possível montar *pipelines* com outros pré-processamentos e outros tipos de modelo sem muita dificuldade.

26/02/2026, 23:15

Engenharia de software para ciência de dados

cod.py

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

model = Pipeline([('StandardScaler', StandardScaler()),
                  ('KNN', KNeighborsClassifier())])

model.fit(X_train, Y_train)
accuracy_score(model.predict(X_test), Y_test)
```

Embora seja um bom exemplo do DIP, já que envolve programar para a abstração, a classe *Pipeline* verifica internamente se os elementos que pretende utilizar contêm os métodos *fit* e *transform*, ou seja, se a abstração está sendo atendida, e alerta no caso de problemas, sem utilizar explicitamente o conceito de interface. Provavelmente essa decisão de projeto foi tomada considerando que as exigências para o seu último elemento são diferentes daquelas para os demais.

Boas práticas de codificação

Com a finalidade de manter seu código organizado e fácil para que outros programadores (ou até você mesmo) possam compreender e dar manutenção, recomenda-se que um bom programador siga boas práticas para escrever seus códigos. Entre essas práticas, está seguir guias de estilo na escrita de código para facilitar sua leitura. Para a linguagem Python, a principal guia de estilos é a PEP-8, cujo principal autor é o criador do Python – Guido Von Rossum.

Além de seguir guias de estilo, é importante que o programador se assegure de que seu código seja de fácil compreensão e manutenção. Nesse contexto, o termo *clean code* (código limpo) se popularizou com o livro de Robert C. Martin *Clean Code: A Handbook of Agile Software Craftsmanship*, publicado em 2008. As boas práticas de um código limpo consistem em instruções direcionadas ao modo como escrevemos nosso código para facilitar sua manutenção.

Estudo complementar

Tanto o guia de estilos quanto as boas práticas de *clean code* estão mais relacionados com a implementação do código orientado a objetos, ou seja, programação, do que com outros aspectos da engenharia de software. Mais detalhes sobre esses assuntos podem ser encontrados no conteúdo extra de introdução a Python e programação orientada a objetos.

Para acessar a disciplina bonus vá até a área de entrada do AVA e clique em Programação Orientada a Objetos.

Controle da qualidade no contexto de *machine learning*

Reprodutibilidade de notebooks

Como vimos, na área de ciência de dados é extremamente comum realizar tarefas como análise exploratória, pré-processamento e avaliação de algoritmos de *machine learning* utilizando programação literária interativa (*notebooks*). Entretanto, a forma interativa de produção do código pode prejudicar sua reprodutibilidade. Em um estudo analisando *notebooks* disponíveis no GitHub, foi identificado que somente 25% dos *notebooks* permitem a execução de todas as suas células e que apenas 10% produzem resultados próximos aos armazenados nos *notebooks* (PIMENTEL *et al.*, 2021).

A partir desse estudo, foram compiladas algumas boas práticas visando à reprodutibilidade dos *notebooks*, que compartilhamos a seguir:

- Usar títulos descritivos com um conjunto restrito de caracteres (A-Z a-z 0-9 . _) e células *markdown* com títulos mais detalhados no corpo.
- Prestar atenção ao final do *notebook* e verificar se as células do final foram executadas e se não cabe nenhuma célula de *markdown* para concluir a análise.
- Abstrair funções, classes e módulos que possam ser importados por diversos *notebooks* e testados externamente.
- Declarar dependências em arquivos apropriados (*requirements.txt*) e fixar a versão dos módulos.
- Usar um ambiente limpo para testar dependências e verificar se todas estão declaradas corretamente.
- Colocar importações no início do *notebook*.
- Usar caminhos relativos para acessar dados no repositório.
- Reexecutar *notebooks* do início ao fim antes de enviá-los para algum repositório.

Estas são práticas intuitivas, mas que podem se mostrar relevantes para melhorar a reprodutibilidade de *notebooks*, o que pode se mostrar essencial principalmente em momentos em que modelos construídos utilizando *notebooks* precisam ser atualizados.

Análise estática

Análise estática é o processo de analisar automaticamente um código com base na sua estrutura sem envolver sua execução, ou seja, de forma estática (WAGNER, 2013). Ferramentas de análise estática são comumente chamadas de *linters*. Essas ferramentas em geral são capazes de verificar se guias de estilo estão sendo seguidos, encontrar determinados padrões de defeitos e analisar aspectos de fluxo de controle e fluxo de dados dos programas. Entretanto, um analisador estático não será capaz de dizer se o programa faz o que ele se propõe a fazer. Ou seja, a análise estática, embora útil, precisa ser complementada com outras formas de controle da qualidade.

Os *linters* mais usados no mercado no contexto do Python são o Flake8 e o Pylint. Ambos verificam o guia de estilos PEP-8, padrões de defeitos e aspectos de fluxo de controle e fluxo de dados dos programas, além de outras opções. Embora haja alguma

sobreposição, não há problema em combinar ferramentas de análise estática. Tanto o Flake8 quanto o Pylint podem ser utilizados com *notebooks*, e é possível encontrar adaptações para seu uso no contexto de *notebooks* (por exemplo, *flake8-nb*).

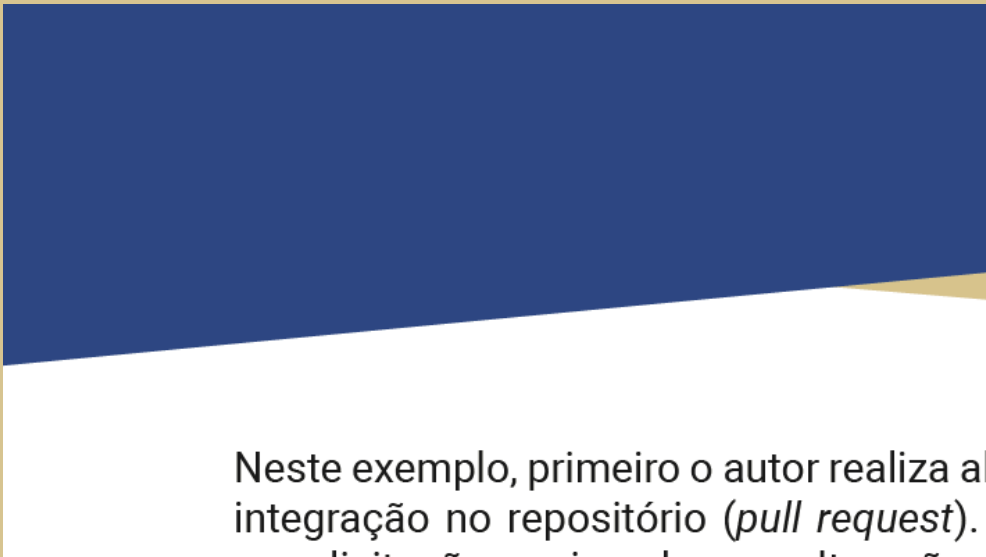
Revisões de código modernas

Revisões de código modernas desempenham um importante papel no controle da qualidade de *software*. Elas permitem que defeitos que o desenvolvedor deixou passar durante o desenvolvimento possam ser corrigidos antes que passem para o repositório de código compartilhado.

Mantendo o foco!

Essencialmente, revisores refletem sobre a qualidade do código de um desenvolvedor, referente a pequenas mudanças no sistema, antes que essas mudanças sejam integradas no repositório de código. Os defeitos encontrados são passados para que o autor possa corrigir e submeter uma nova versão. O que caracteriza as revisões de código como 'modernas' (embora o termo tenha sido definido há mais de 10 anos) é a sua informalidade e sua integração leve no processo de desenvolvimento, permitindo obter *feedback* rápido sobre mudanças propostas no código.

Veja, a seguir, como o processo de revisão de código moderna ocorre no desenvolvimento.



Interativo

Descrição do interativo

Diversas empresas, como Google (SADOWSKI *et al.*, 2018) e Microsoft (BACCHELLI; BIRD, 2013), têm reportado benefícios da

aplicação de revisões de código modernas, e a prática tem-se disseminado como uma das boas práticas da engenharia de *software*. De fato, elas permitem identificar não somente defeitos, como também vulnerabilidades de segurança e problemas estruturais no código. Além disso, tendem a tornar o código mais simples de ser mantido e promovem a transferência de conhecimento na equipe de desenvolvimento.

Naturalmente, essa prática de controle da qualidade pode também ser aplicada no desenvolvimento de sistemas de *software* inteligentes envolvendo *machine learning*, já que a natureza do código sendo construído não afeta diretamente a prática. Basta utilizar apropriadamente ferramentas capazes de lidar com solicitações de mudança (como o Jira e o Microsoft Azure Devops) e controle de versões (como o Git) para o código sendo construído. Normalmente, o ambiente para viabilizar as revisões de código modernas é preparado pelo analista de infraestrutura e DevOps.

Teste de sistemas de *software* inteligentes

O teste de *software* é realizado em diferentes fases ao longo do desenvolvimento: teste unitário, teste de integração, teste de sistema e teste de aceitação.

Fique ligado!

O teste unitário se refere à fase do processo de teste em que se testam as menores unidades de *software* desenvolvidas. O teste de integração envolve, a partir dos módulos testados no nível de unidade, testar a estrutura de programa que foi determinada pelo projeto. O teste de sistema envolve executar o sistema sob o ponto de vista de seu usuário final, varrendo as funcionalidades em busca de falhas. Por fim, o teste de aceitação é realizado pelo cliente.

Além disso, como vimos, no contexto de sistemas de *machine learning*,o uso de teste A/B (experimentação contínua) em produção é muito comum. Falaremos rapidamente de cada uma dessas fases a seguir, com ênfase em aspectos relacionados com *machine learning*.

Antes de iniciar a discussão, é importante ressaltar que teste de *software* é uma disciplina ampla e que tem material muito rico disponível na literatura de engenharia de *software*, além do escopo desta disciplina. Para uma visão mais rica sobre a disciplina de teste de *software* de maneira geral, recomendamos consultar os livros de Aniche (2022) ou Delamaro e colaboradores (2016).



Em relação ao teste de componentes de *machine learning*, independentemente da fase de teste, para testar as predições dos modelos é preciso escolher métricas adequadas e *thresholds* (valores limite) aceitáveis para o desempenho do modelo, em função dos requisitos definidos, para determinar se o teste passou ou falhou. Requisitos de desempenho para um modelo de classificação poderiam envolver, por exemplo, acurácia acima de 85%. Requisitos de desempenho para um modelo de regressão poderiam envolver, por exemplo, R2 score acima de 0,7. Além de testar os requisitos de desempenho do modelo, dependendo dos requisitos levantados, outras características referentes ao modelo podem precisar ser verificadas, incluindo o tamanho do modelo, o tempo de treinamento, o tempo de inferência, entre outras. Ainda em relação ao componente de *machine learning*, além das características relacionadas à perspectiva do modelo, não podemos esquecer de testar as características relevantes das perspectivas de objetivos, experiência de usuário, infraestrutura e dados (KALINOWSKI *et al.*, 2023).

Em relação aos testes unitários, idealmente as classes são testadas externamente aos *notebooks*. PyTest é o principal *framework* para teste unitário em Python. É importante ressaltar que os testes de código orientado a objetos serão mais eficientes se forem construídos utilizando técnicas de teste estruturais, incluindo técnicas de fluxo de controle e de fluxo de dados da engenharia de *software* tradicional. Entretanto, esses testes não são suficientes no contexto de sistemas de *machine learning*, em que o algoritmo pode estar correto, mas ainda assim o modelo pode ter aprendido comportamentos errados a partir dos dados a que foi submetido.

Assim, é preciso monitorar de perto a qualidade dos dados. Para testar a predição dos modelos, é comum que se utilize um conjunto de dados validados pelo especialista de domínio em relação à sua qualidade (representatividade, corretude, consistência, credibilidade, etc.), comumente conhecido como *golden dataset*. **Para mais detalhes sobre a qualidade dos dados, recorra ao tópico de requisitos de sistemas de software inteligentes.**

Certamente, o menor desafio nos testes unitários de código de *machine learning* será aprender a utilizar a sintaxe do PyTest. O PyTest é um *framework* para teste unitário prático e simples de utilizar que identifica automaticamente os arquivos nomeados como **test_*.py** ou ***_test.py** e, dentro deles, as funções iniciadas com

`test_` e então as executa, reportando os resultados em função da avaliação de comandos `assert`.

O código a seguir, contido em um arquivo fictício `test_load.py` tem uma função de teste, chamada `test_load_data`, que verifica, por meio de uma assertiva, se uma classe responsável por carregar o *dataset* (*loader*) está realizando a carga de todas as linhas do *dataset*, verificando se o seu tamanho (número de linhas) corresponde a 150.

```
from loader import Loader

def test_load_data():
    url_dados = ('https://archive.ics.uci.edu/'
                 'ml/machine-learning-databases/iris/iris.data')
    atributos = ['comprimento_sepala', 'largura_sepala',
                 'comprimento_petala', 'largura_petala',
                 'especie']

    loader = Loader()
    dataset = loader.load_data(url_dados, atributos)

    assert len(dataset) == 150
```

Instrumentar o seu código com testes unitários é uma boa prática, pois permitirá identificar o quanto antes se alguma ação realizada fez com que o sistema (ou o modelo) não atenda mais ao que se espera dele.

>

O teste de integração permitirá avaliar se as unidades sendo desenvolvidas pelos diferentes desenvolvedores funcionam em conjunto como parte do produto contido no repositório integrado. É comum que, para esse tipo de teste, o mesmo *golden dataset* seja utilizado.

>

Testes de integração tendem a se beneficiar de práticas de integração contínua e automatização, como a geração automática de *builds* e testes automatizados. Falaremos mais sobre integração contínua no próximo tópico desta aula.

O teste de sistema envolve executar o sistema sob o ponto de vista de seu usuário final, verificando tanto requisitos funcionais quanto requisitos não funcionais do sistema como um todo (como segurança e usabilidade, entre outros). Nessa fase, é interessante que se trabalhe com dados mais próximos aos dados reais de produção, evitando surpresas nos testes de aceitação.

O teste de aceitação pode ser realizado no ambiente de desenvolvimento simulando da melhor forma possível o ambiente de produção (teste *alpha*) ou, idealmente, no ambiente final de produção (teste *beta*). Eles são realizados pelo cliente, e, independentemente de o teste ser *alpha* ou *beta*, neste momento estamos falando de testes com dados de produção.

No contexto de sistemas de *machine learning*, o uso de testes A/B (experimentação contínua) realizados em produção é muito comum. Nesses testes se comparam duas alternativas de solução em produção com diferentes amostras de usuários para avaliar (estatisticamente, com testes de hipótese, se possível) se uma solução inovadora de fato está trazendo ganhos quando comparada à solução atualmente em vigor. Quando se trata de experimentar uma solução de inteligência computacional substituindo outra, é comum que se faça um *shadow deployment*, para que ambas possam executar simultaneamente e os resultados possam ser comparados, permitindo chegar a uma conclusão sobre a solução que traz os melhores resultados para o negócio.

Gerência de configuração e Devops

Gerência de configuração

Gerência de configuração é o processo de identificar, organizar e controlar modificações no *software* sendo construído. Um projeto de desenvolvimento de *software* produz diversos itens, como programa, documentação e dados. Esses conjuntos de itens são chamados, coletivamente, de configuração do *software*.

Nesse contexto, um conceito importante é o de **baseline**, que se refere a uma configuração do *software*, em um ponto discreto do tempo, que foi formalmente revisada e aceita.

Mantendo o foco!

A *baseline* serve como base para os passos posteriores do desenvolvimento. No contexto ágil, é comum que se estabeleçam *baselines* após cada uma das *sprints* de desenvolvimento, contendo as novas funcionalidades entregues e aprovadas naquela *sprint*. Na prática, as *baselines* são estabelecidas utilizando o recurso de *tags* de sistemas de controle de versões, como o Git. O estabelecimento de *baselines* em sistemas de controle de versões permite reproduzir uma versão anterior do sistema quando isso for desejado.

Por fim, dois outros conceitos importantes de gerência de configuração são o **build** e a **release**. Um **build** representa uma versão não necessariamente completa do sistema em desenvolvimento, mas com certa estabilidade, que pode ser implantada. Um *build* inclui não só código compilado, mas também a documentação, arquivos de configuração, base de dados, etc. Já a **release** se refere a um **build** entregue ao cliente (interno ou externo). No contexto dos métodos ágeis, busca-se realizar uma *release* a cada *sprint*. Assim, uma *release* implica no estabelecimento de uma nova *baseline*, permitindo que se possa retornar à exata versão entregue ao cliente em momentos futuros.

Uma boa gerência de configuração requer a integração de ferramentas de gestão de tarefas e de solicitações de mudanças (como o JIRA ou o Azure DevOps) com ferramentas de controle de versões (como o Git ou o SVN). Isso permitirá saber exatamente quais solicitações de mudança foram atendidas entre uma versão e outra do produto (*release notes*) e quais foram as exatas alterações feitas nos arquivos para implementar cada uma das solicitações de mudança.

Gerência de configuração está fortemente ligada com o conceito de DevOps, que detalharemos a seguir.



DevOps envolve alinhar o desenvolvimento do *software* com a operação, permitindo aumentar a capacidade de resposta a mudanças por meio de entregas rápidas e de alta qualidade para a operação.

É comum que, para isso, se estabeleça um *pipeline* de integração contínua (*continuous integration* – CI) e implantação contínua (*continuous deployment* – CD), que permitirá que as mudanças sendo desenvolvidas sejam rapidamente integradas no ambiente de desenvolvimento, além de migrar versões aprovadas com segurança entre os ambientes de desenvolvimento, teste, homologação e produção.

A preparação desse *pipeline* é normalmente responsabilidade de um analista de infra/DevOps e comumente utiliza ferramentas como AWS CodePipeline, Azure Pipelines e GitLab CI/CD, que buscam automatizar o processo de geração de *builds* e a implantação (*deploy*). A seguir, abordaremos três práticas que se mostram importantes no contexto de DevOps e que engenheiros de *software* e *machine learning* deveriam conhecer: **controle de versões** , **integração contínua** e **implantação contínua**.

Controle de versões

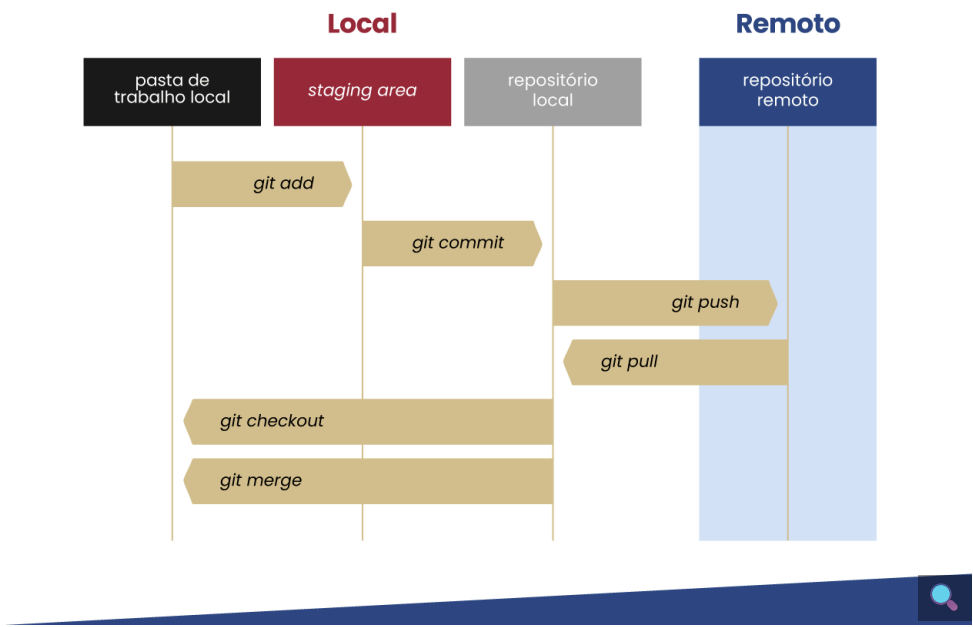
As ferramentas de controle de versões buscam manter todos os arquivos em um repositório central e controlar o acesso a esse repositório, de modo a garantir a consistência dos artefatos. O uso de uma ferramenta de controle de versões é fundamental quando se busca desenvolver um *software* em equipe. Hoje, a ferramenta mais utilizada para o controle de versões é o Git.

Aprenda Mais!

Explicaremos os conceitos básicos que permitirão utilizar o Git com o devido entendimento. Para um referencial completo de comandos do Git, sugerimos recorrer à documentação oficial do Git, que se encontra disponível [aqui](#).

Conforme ilustrado a seguir, o Git funciona com dois repositórios, um local e um remoto. Além disso, existe a área intermediária, conhecida como *staging area*, que armazena os arquivos que se pretende versionar. Os arquivos da sua pasta de trabalho local podem ser

adicionados à *staging area* utilizando o comando *git add*. Alterações em arquivos adicionados na *staging area* podem ser passados para o repositório local utilizando o comando *git commit*. Alterações no repositório local podem ser passados para o repositório remoto utilizando o comando *git push*. Alterações feitas no repositório remoto por outros desenvolvedores podem ser obtidas utilizando o comando *git pull*. É possível fazer cópias do repositório local para uma pasta de trabalho local utilizando o comando *git checkout*. O comando *git merge* permitirá integrar mudanças do seu repositório local para a sua pasta de trabalho local.



Além desses comandos, dois conceitos importantes nesse contexto são o de *branch* e *merge*.



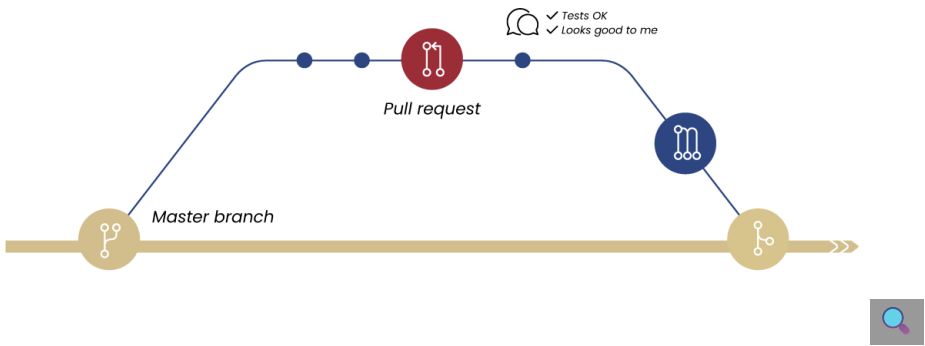
Um *branch* representa um fluxo alternativo para atualização de versões de itens de configuração. *Branches* normalmente se originam de correções em versões anteriores. Dessa forma, faz-se um *pull* do repositório remoto daquela *baseline*, um *checkout* do repositório local, para então criar um *branch* e realizar as correções de forma que não afete outras versões. Caso a correção se aplique também às versões futuras, será preciso realizar um *merge* para a integração da mudança no fluxo principal.

O *merge* nada mais é do que a unificação de diferentes versões de um mesmo item de configuração, ou seja, a integração dos itens de configuração de um *branch* com os itens de configuração do fluxo principal (conhecido como *master branch*).



Muitas vezes, a integração de um *branch* alternativo em um *branch master* é precedida de um *pull request*, ou seja, uma solicitação de integração. O uso de *pull requests* permite integrar nesse processo as revisões de código modernas, que detalhamos anteriormente. Veja, a seguir, essa integração.

Um *branch* alternativo foi criado, e, após dois *commits* de alterações nesse *branch*, um *pull request* foi solicitado. As alterações são, então, revisadas antes que ocorra o *merge*, que integra as alterações, caso aprovadas, no fluxo principal (*master branch*).



Integração contínua

A integração contínua consiste na frequente integração das alterações sendo feitas pelos desenvolvedores no repositório remoto e na geração frequente (pelo menos diária) de *builds* do sistema.



É comum que esses *builds* sejam submetidos a testes unitários e de integração automatizados, permitindo que problemas de integração passem a ser encontrados logo que introduzidos, na maioria dos casos. A integração contínua é considerada uma das melhores práticas do desenvolvimento de *software*. Para uma estratégia eficiente de integração contínua, a geração de *builds* deve ser automatizada e realizada com frequência adequada.

Implantação contínua

Enquanto com a integração contínua alterações são frequentemente integradas no *branch* principal, quando se usa implantação contínua (*continuous deployment*), as alterações no *branch* principal são rapidamente levadas para a produção. Isso requer um conjunto de testes automatizados para que essa migração automática para a produção possa ser feita com segurança. Para isso, algumas vezes ao dia, o servidor de integração contínua realiza testes mais exaustivos, incluindo testes de sistema, com as alterações que ainda não entraram em produção. Se todos os testes passarem, as alterações são automaticamente implantadas em produção e disponibilizadas para os usuários.

Um relato de sucesso amplamente conhecido de implantação contínua é o da estratégia DevOps reportada pelo Facebook (atual Meta) (FEITELSON; FRACHTENBERG; BECK, 2013), que permite à empresa, que até hoje é referência no uso de boas práticas de DevOps, implantar novos desenvolvimentos para que os usuários possam utilizar assim que esses desenvolvimentos estiverem concluídos. Alinhado ao que vimos nesta aula, o artigo de Feitelson, Frachtenberg e Beck (2013) reforça as revisões de código modernas e testes automatizados como fatores-chave para o sucesso de estratégias de DevOps.

Referências

Clique aqui para acessar as referências e créditos desta aula.

